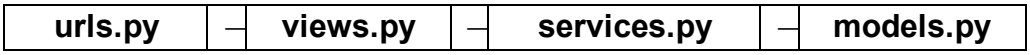
Resultado de aprendizaje — Unidad 1

Proyecto Django REST: precios finales por categoría

Arquitectura del proyecto

El proyecto usa la estructura MVT de Django y está organizado en capas. La ruta recibe la petición, la vista revisa el acceso con JWT, el servicio calcula el precio y el modelo obtiene los datos del producto. Así, el descuento queda en un solo lugar y la vista es más fácil de entender.



El recorrido empieza en /api/products/<id>/precio-final/. La vista pide el resultado al servicio y el servicio consulta el producto y su categoría; después regresa el precio final en formato JSON.

Strategy: descuento según la categoría

En products/services.py se definieron tres formas de calcular el precio: 10 % de descuento para Electrónica, 20 % para Ropa y sin descuento para las demás categorías. El servicio elige la opción que corresponde al nombre de la categoría y calcula con dos decimales.

Captura 1. Estrategias de descuento en services.py

Evidencia: estrategia de descuento

Fragmento del archivo real del proyecto.

products/services.py

```python
from decimal import Decimal
from unicodedata import normalize

from django.shortcuts import get_object_or_404

from .models import Product


class EstrategiaDescuento:
    porcentaje = 0
    factor = Decimal('1.00')

    def calcular(self, precio):
        return (Decimal(precio) * self.factor).quantize(Decimal('0.01'))


class DescuentoElectronica(EstrategiaDescuento):
    porcentaje = 10
    factor = Decimal('0.90')


class DescuentoRopa(EstrategiaDescuento):
    porcentaje = 20
    factor = Decimal('0.80')


class SinDescuento(EstrategiaDescuento):
    pass


def obtener_estrategia(categoria_nombre):
    nombre = normalize('NFKD', categoria_nombre).encode('ascii', 'ignore').decode().casefold()
    return {
        'electronica': DescuentoElectronica,
        'ropa': DescuentoRopa,
    }.get(nombre, SinDescuento)()


def obtener_precio_final(pk):
    producto = get_object_or_404(Product.objects.select_related('category'), pk=pk)
    estrategia = obtener_estrategia(producto.category.name)
    return {
        'producto': producto.name,
        'categoria': producto.category.name,
```

Factory Method: categoría predeterminada

El método crear_con_categoria_default crea un producto cuando no se indica una categoría. Primero busca o crea “Sin categoría” y luego guarda el producto con esa relación. Esto evita repetir esos pasos cada vez que se necesite un producto así.

Captura 2. Método en models.py

Evidencia: creación con categoría predeterminada

Fragmento del archivo real del proyecto.

products/models.py

```
40      @classmethod
41      def crear_con_categoria_default(cls, name, description, price):
42          categoria, _ = Category.objects.get_or_create(name='Sin categoría')
43          return cls.objects.create(
44              name=name,
45              description=description,
46              price=price,
47              category=categoria,
48          )
```

Facade: consulta del precio final

El endpoint /api/products/<id>/precio-final/ está protegido con JWT. La vista hace una sola llamada al servicio y entrega el resultado; por eso funciona como una entrada sencilla al cálculo completo. La ruta también está disponible bajo /api/v1/.

Prueba con dos categorías

Se consultó el endpoint con un token de acceso para dos productos reales del proyecto. Mouse cuesta 250.00 y, al ser de Electrónica, queda en 225.00; Playera cuesta 180.00 y, al ser de Ropa, queda en 144.00. Las dos peticiones respondieron con código 200.

Captura 3. Respuestas del endpoint con JWT



Evidencia: /precio-final/ con JWT

Respuestas reales obtenidas con un token temporal de acceso.

GET http://127.0.0.1:8000/api/products/1/precio-final/
HTTP 200


```
{
  "producto": "Mouse",
  "categoria": "Electrónica",
  "precio_original": "250.00",
  "descuento_porcentaje": 10,
  "precio_final": "225.00"
}
```


GET http://127.0.0.1:8000/api/products/3/precio-final/
HTTP 200


```
{
  "producto": "Playera",
  "categoria": "Ropa",
  "precio_original": "180.00",
  "descuento_porcentaje": 20,
  "precio_final": "144.00"
}
```

Pruebas del endpoint

Se agregaron dos pruebas nuevas. La primera consulta dos productos con acceso autorizado y comprueba el precio de cada uno; la segunda consulta sin token y espera el código 401. Las pruebas anteriores también siguen funcionando.

Captura 4. Pruebas nuevas en tests.py

Evidencia: dos pruebas del endpoint

Fragmento del archivo real del proyecto.

products/tests.py

```
110     def test_precio_final_con_token_para_dos_categorias(self):
111         productos = [
112             (self.mouse.pk, 'Electrónica', Decimal('225.00')),
113             (Product.objects.get(name='Playera').pk, 'Ropa', Decimal('144.00')),
114         ]
115         for pk, categoria, total in productos:
116             with self.subTest(categoria=categoria):
117                 response = self.client.get(f'/api/products/{pk}/precio-final/')
118                 self.assertEqual(response.status_code, 200)
119                 self.assertEqual(response.data['categoria'], categoria)
120                 self.assertEqual(Decimal(response.data['precio_final']), total)
121
122     def test_precio_final_sin_token_devuelve_401(self):
123         cliente_sin_token = APIClient()
124         response = cliente_sin_token.get(
125             f'/api/products/{self.mouse.pk}/precio-final/'
126         )
127         self.assertEqual(response.status_code, 401)
```

Captura 5. Salida de python manage.py test

Salida de python manage.py test

Registro real de la ejecución de pruebas del proyecto.

```
PS C:\Users\eberh\django-project> python manage.py test
Creating test database for alias 'default'...

............
----------------------------------------------------------------------
Ran 12 tests in 2.023s

OK
Destroying test database for alias 'default'...

Found 12 test(s).
System check identified no issues (0 silenced).
```

Tabla de patrones de diseño

Los tres patrones se aplicaron en partes distintas del proyecto. Cada uno ayuda a ordenar una tarea concreta: crear productos, elegir descuentos o responder desde la API.

Nombre	Categoría	Evidencia	Explicación
Factory Method	Creacional	Método de Product en models.py	Crea el producto en “Sin categoría”.
Strategy	Comportamiento	Clases de descuento en services.py	Elige 10 % o 20 % según la categoría.
Facade	Estructural	precio_final() en views.py	Da una sola entrada al cálculo del servicio.